

Reinforcement Learning

H. M. Samadhi Chathuranga Rathnayake



What is Reinforcement Learning?

- Think about the situation when you are training a dog. We can't direct the dog because he doesn't speak our language.
- We instead use a different approach.
- The dog tries to respond in a variety of ways as we imitate a situation (or a cue).
- We give the dog treats if their answer is what we were hoping for.
- Guess what, the dog performs the same action even more enthusiastically the following time it encounters the same circumstance in hopes of receiving more food.
- That's like learning "what to do" from favorable experiences.
- Dogs similarly have a tendency to pick up on bad behavior when faced with it.



What is Reinforcement Learning?



Our dog is a "**agent**" who is in contact with the outside world. The setting might be in your home, near you.



The circumstances they run into are comparable to a **state**. A state might be your dog standing as you speak in a particular tone in your living room.



Your dog moves from standing to sitting as a result of our agents' actions, which transfer it from one "**state**" to another.

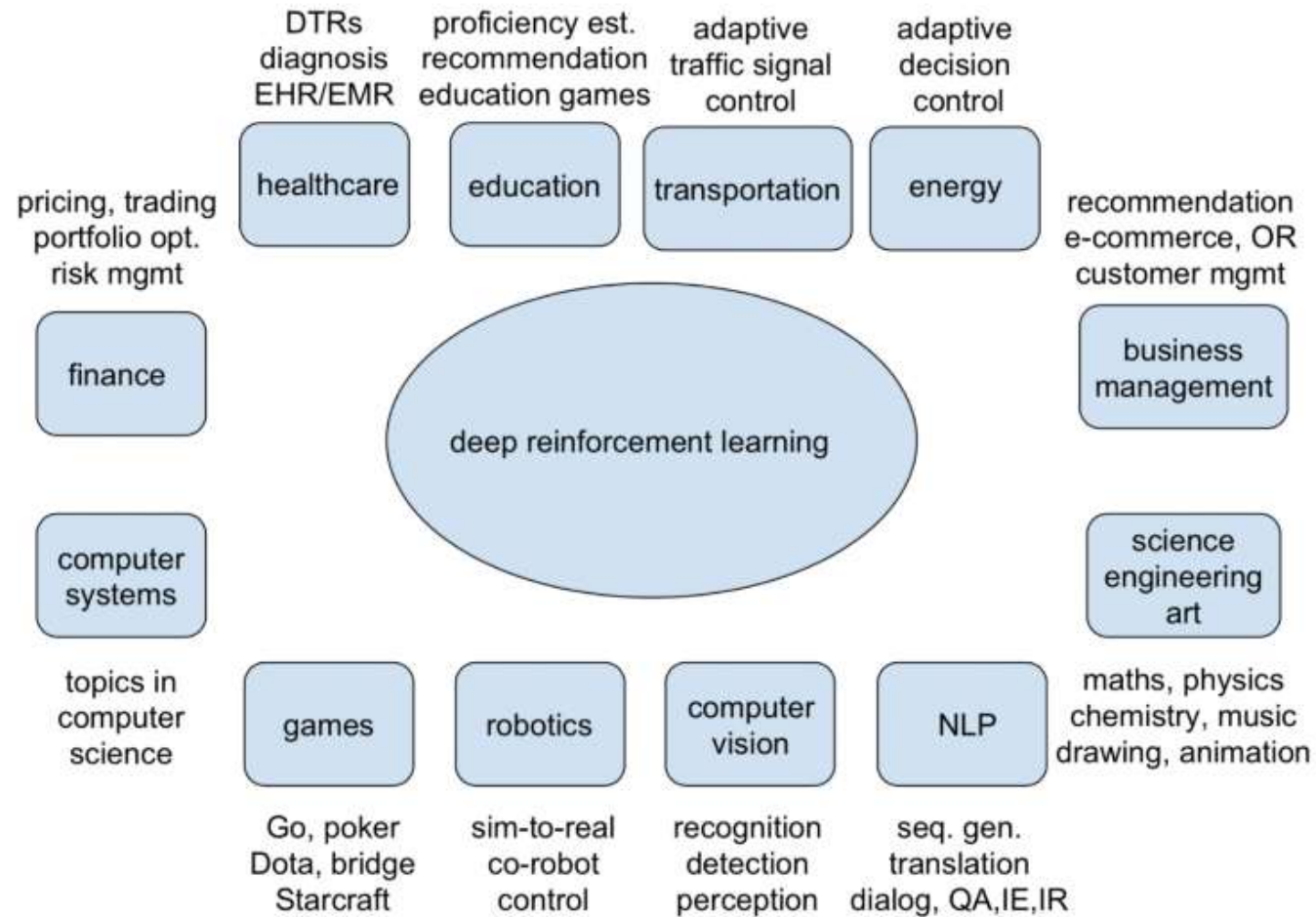


Following the shift, they might be given a **reward** or a **penalty**. They get a treat from you! Or a "No" as a penalty.



The method of selecting a course of action given a state in the hope of better results is known as a **policy**.

Applications of Reinforcement Learning





Agent

- The agent is the person who makes decisions based on rewards and penalties in reinforcement learning.
- Take a cricket batsman as an example. If he misses, he loses a point.
- He tries to hit the ball.
- He receives a reward if he hits the ball.
- He will learn how to play that specific ball as a result of these positive and bad experiences. The batsman in this instance is a representative.

Environment

- The Agent's environment in reinforcement learning is the setting in which it interacts and lives.
- The agent can interact with the environment by taking certain actions, but those actions cannot change the environment's dynamics or laws.
- This means that if humans were to influence the environment of the world, we would be constrained by the planet's physical laws.
- Our activities can influence the environment, but they cannot alter the physics of our world.

Deterministic vs Stochastic Environment

- Deterministic Environment
 - In a deterministic environment, the next state of the environment can always be determined based on the current state and the agent's action.
 - For instance, if the agent steers left when operating a vehicle, the vehicle will only go in that direction.
 - In a perfect world, steering the car left will never cause it to move right; instead, it will always move left, according to determinism.
- Stochastic Environment
 - Consider the possibility that our agent's experience behind the wheel is not flawless.
 - There is a little chance that a break applied by an agent will not work and the car will not stop.
 - Similar to this, there is a slight risk that the car will just stop when the agent tries to accelerate it.
 - The next state of the car cannot always be predicted in this setting based on its current state and the agent's action.

Single Agent vs Multi-Agent Environment

- Single Agent Environment
 - In a single agent environment, there is only one agent that exists and interacts with the environment.
 - For instance, it is a single agent environment if just one agent is operating the vehicle as it travels from point A to point B.
- Multi-Agent Environment
 - In a multi-agent environment, there are more than one agents that are interacting with the environment.
 - Therefore, in our example, a multi-agent environment exists if there are many cars being driven by various agents.

Discrete vs Continuous Environment

- Discrete Environment
 - In a discrete environment, the action space of the environment is discrete in nature.
 - For example, consider a grid world environment where the available action space is discrete like [left, right, up, down].
- Continuous Environment
 - In a continuous environment, the action space of the environment is continuous in nature.
 - A self-driving car world is an example of a continuous environment.
 - Here the action space of the environment consists of continuous action like car's speed, acceleration, steering rotation, etc.

Episodic vs Sequential Environment

- Episodic Environment
 - In an episodic environment, the agent's actions are confined to the particular episode only and not on any previous actions.
 - For example, in a game of cups and ball where the task is to guess the correct cup out of three cups having the ball, the agent's action is independent of its earlier attempts or episodes.
- Sequential Environment
 - In a sequential environment, the agent's actions are connected with the previous actions it took.
 - For example, in Chess, the current state of the game was reached due to all the game-playing actions the agent took.
 - And all the future actions in the game are also going to be dependent on its previous history of sequences.

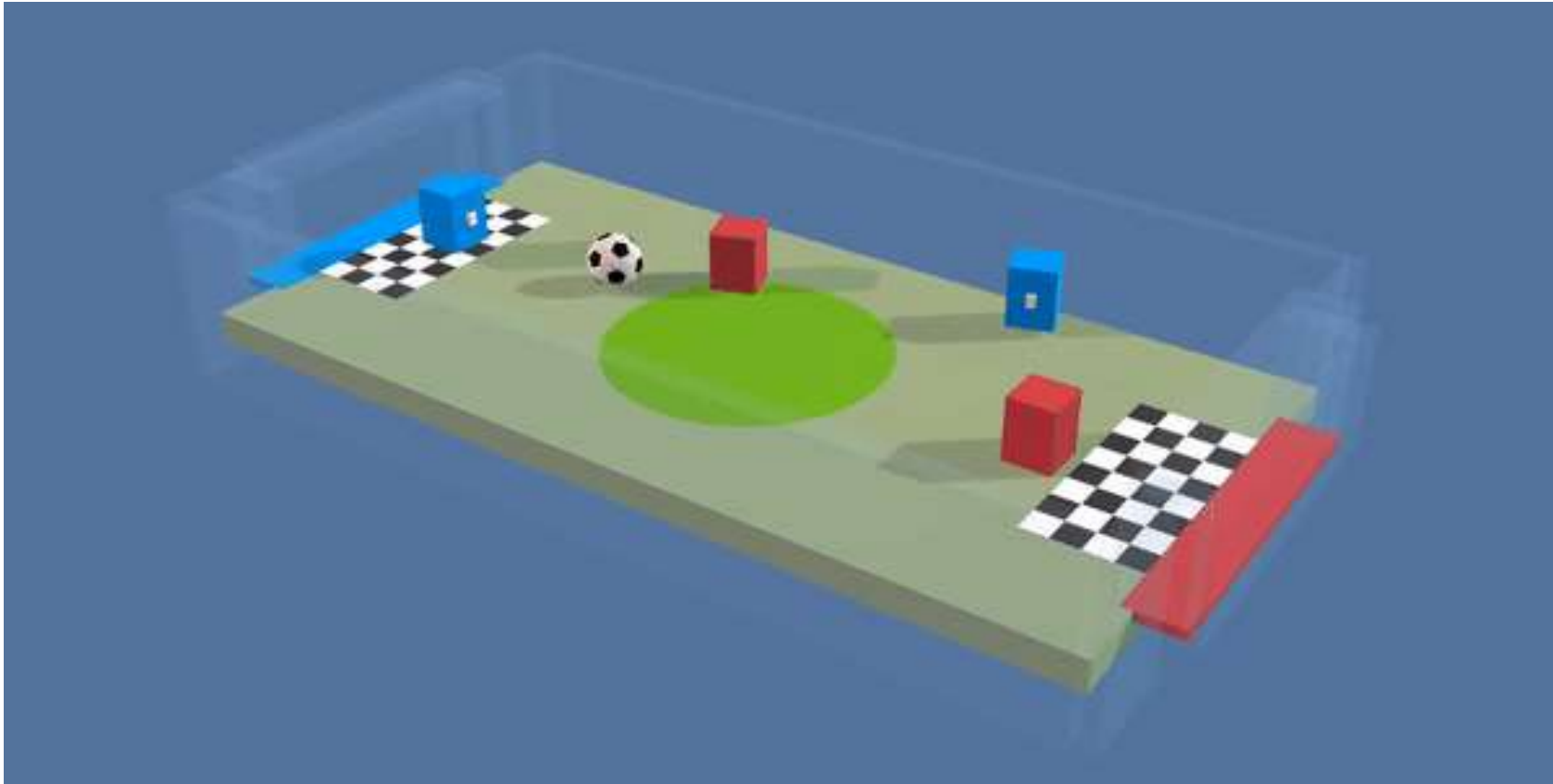
Fully Observable vs Partially Observable Environment

- Fully Observable Environment
 - In a fully observable environment, the agent is always aware of the complete state of the environment at any given point in time.
 - In the game of chess, the agent can always see the complete position of itself and its opponent on the board. Hence it is a fully observable environment.
- Partially Observable Environment
 - In a partially observable environment, the agent cannot always see the complete state of the environment at any given point in time.
 - In the game of poker, the agent cannot see the hands of the opponent for the most part of the game, and hence it is a partially observable environment.

State

- A state in reinforcement learning is a representation of the current environment that the agent is in.
- This state can be observed by the agent, and it includes all relevant information about the environment that the agent needs to know in order to make a decision.
- This state changes dynamically.

State of The Environment Changes



Reward

- The agent's goal is specified by this component.
- In essence, we divide the entire process into time steps and assume that the agent does some action during each time step.
- Every action results in a change in the environment's state and a numerical reward that is given to the agent.
- This figure is influenced by the agent's behavior in time step t as well as the environment's condition in time interval t .
- Agent can thus affect reward in two different ways.
- It can improve its reward either directly through its behaviors or indirectly by altering the surroundings.
- In essence, rewards determine whether a behavior is good or poor, and the agent aims to maximize the reward over time.

Policy

- The fundamental component of reinforcement learning is the policy.
- It specifies the action the agent will take in a specific environment state.
- In essence, it links an agent's activities to environmental states.

Value Function

- The agent sees reward as an instant desire to be in a particular state.
- In other words, if we compare it to humans, the immediate objective is the reward.
- Unlike reward, value function defines a long-term goal for the agent.
- The value of the state represents the amount of reward agent can accumulate in the future, starting from that state.
- The value represents the prediction of rewards.
- In essence, the agent observes it as long-term desirability to be in a certain state.
- From the perspective of the agent, incentives are the main goal and values are secondary.
- This results from the environment's stochastic nature.
- However, judgments regarding the subsequent course of action are always made in light of the ideals.
- As a result, the agent is constantly attempting to enter the state with the highest value because doing so will ultimately yield greater rewards.

Markov Decision Process

- By taking action in each discrete time-step, the agent engages with the environment.
- The environment will alter its status in response to that action and provide some sort of monetary compensation.
- Utilizing the value function, the agent will strive to formulate a strategy that will maximize the payoff.
- Actually, reinforcement learning is attempting to address this issue.

Markov Decision Process

- We employ the **Markov Decision Processes (MDPs)** framework to represent this mathematically.
- Using this paradigm, nearly all reinforcement learning issues may be codified.

Markov Decision Process

- Markov Decision Process has four elements (S, A, P_a, R_a)
- S – Represents the **set of states**. At each time step t , the agent gets the environment's state – S_t , where $S_t \in S$.
- A – Represents the **set of actions** that the agent can foretake. At each time step t , based on the received state S_t , the agent makes the decision to perform an action – A_t , where $A_t \in A(S_t)$. $A(S_t)$ represents a set of possible actions in the state S_t .
- P_a – Represents the **probability** that action in some state s will result in the time step t , will result in the state s' in the time step $t+1$.

$$P_a(s, s') = P(s' = S_{t+1} | S_t = s, A_t = a)$$

- R_a – Or more precisely $R_a(s, s')$, represents **expected reward** received after going from state s to the state s' , as a result of action a .

Markov Decision Process

- For easy understanding of equations, we will define some notations.
- We denote a set of states as $\mathbf{S}$, a set of actions as $\mathbf{A}$, and a set of rewards as $\mathbf{R}$.
- At each time step $t = 0, 1, 2, \dots$, some representation of the environment's state $\mathbf{S}t \in \mathbf{S}$ is received by the agent.
- According to this state, the agent selects an action $\mathbf{A}t \in \mathbf{A}$ which gives us the state-action pair $(\mathbf{S}t, \mathbf{A}t)$.
- In the next time step $t+1$, the transition of the environment happens and the new state $\mathbf{S}t+1 \in \mathbf{S}$ is achieved.
- At this time step $t+1$, a reward $\mathbf{R}t+1 \in \mathbf{R}$ is received by the agent for the action $\mathbf{A}t$ taken from state $\mathbf{S}t$.

Markov Decision Process

- As we mentioned above that the goal of the agent is to maximize the cumulative rewards, we need to represent this cumulative reward in a formal way to use it in the calculations.
- We can call it as Expected Return and can be represented as,

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

Markov Decision Process

- The above equation works for episodic tasks.
- For continuing tasks, we need to update this equation as we don't have a limit of time step T in continuing tasks.
- Discount factor γ is introduced here which forces the agent to focus on immediate rewards instead of future rewards.
- The value of γ remains between 0 and 1.

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \\ &= \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \end{aligned}$$

Markov Decision Process

- Policy is a function that maps a given state to probabilities of selecting each possible action from the given state.
- If at time t , an agent follows policy π , then $\pi(a | s)$ becomes the probability that the action at time step t is $A_t=a$ if the state at time step t is $S_t=s$.
- The meaning of this is, the probability that an agent will take an action a in state s is $\pi(a | s)$ at time t with policy π .

Markov Decision Process

- The value functions are nothing more than a straightforward evaluation of how desirable it is for an agent to be in a particular state or to carry out a particular action in a particular state.
- Value functions come in two varieties, which are detailed below:
 1. State Value Function
 2. Action Value Function

Markov Decision Process

- The state-value function for policy π denoted as v_π determines the goodness of any given state for an agent who is following policy π .
- This function gives us the value which is the expected return starting from state s at time step t and following policy π afterward.

$$\begin{aligned} v_\pi(s) &= E_\pi[G_t \mid S_t = s] \\ &= E_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right] \end{aligned}$$

Markov Decision Process

- The action-value function mentioned above determines the goodness of the action taken by the agent from a given state for policy π .
- This function gives the value which is the expected return starting from state s at time step t , with action a , and following policy π afterward.
- The output of this function is also called as Q-value where q stands for Quality. Note that in the state-value function, we did not consider the action taken by the agent.

$$\begin{aligned} q_{\pi}(s, a) &= E_{\pi}[G_t \mid S_t = s, A_t = a] \\ &= E_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right] \end{aligned}$$

Markov Decision Process

- The policy π can be considered better than or same as the policy π' if the expected return of policy π is greater than or equal to the expected return of policy π' for all states.
- Such policy π is called Optimal Policy.

$$\pi \geq \pi' \text{ if and only if } v_{\pi}(s) \geq v_{\pi'}(s) \text{ for all } s \in \mathcal{S}.$$

Markov Decision Process

- The optimal policy should have an associated optimal state-value function or action-value function.

$$v_*(s) = \max_{\pi} v_{\pi}(s)$$

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

- The optimal state-value function v_* yields the largest expected return feasible for each of the state s with any policy π .
- And the optimal action-value function q_* yields the largest expected return feasible for each state-action pair (s, a) with any policy π .

Bellman Optimality Equation

- The optimal action-value function q_* satisfies the below-given equation. This is a fundamental property of the optimal action-value function.

$$q_*(s, a) = E \left[R_{t+1} + \gamma \max_{a'} q_*(s', a') \right]$$

Bellman Optimality Equation

- This equation tells that, at time t , for any state-action pair (s, a) , the expected return from starting state s , taking action a , and with the optimal policy afterward will be equal to the expected reward R_{t+1} we can get by selecting action a in state s , in addition with the maximum of “expected discounted return” that is achievable of any (s', a') where (s', a') is a potential next state-action pair.
- Considering the agent following an optimal policy, the latter state s' will be the state from we can take the best possible next action a' at time $t+1$.
- This seems somewhat difficult to understand.
- The essence is that this equation can be used to find optimal q^* in order to find optimal policy π and thus a reinforcement learning algorithm can find the action a that maximizes $q^*(s, a)$.
- That is why this equation has its importance.

Bellman Optimality Equation

- The Optimal Value Function is recursively related to the Bellman Optimality Equation.
- The above property can be observed in the equation as we find $q^*(s', a')$ which denotes the expected return after choosing an action a in state s which is then maximized to gain the optimal Q-value.

Q - Learning

- During the training of an agent in order to determine the optimal policy, we need to update the Q-value (Output of the Action-Value Function) iteratively.
- The Q-learning algorithm iteratively updates the Q-values for each state-action pair using the Bellman Optimality Equation until the Q-function converges to the optimal Q-function, q^* .
- This process is called Value-Iteration.

Q - Learning

- To make the Q-value eventually converge to an optimal Q-value q^* , what we have to do is for the given state-action pair, we have to make the Q-value as near as we can to the right-hand side of the Bellman Optimality Equation.
- For this, the loss between the Q-value and the optimal Q-value for the given state-action pair will be compared iteratively, and then each time we encounter the same state-action pair, we will update the Q-value, again and again, to reduce the loss.
- The loss can be given as $q^*(s, a) - q(s, a)$.

$$q^{new}(s, a) = (1 - \alpha) \underbrace{q(s, a)}_{\text{old value}} + \alpha \overbrace{\left(R_{t+1} + \gamma \max_{a'} q(s', a') \right)}^{\text{learned value}}$$

Q - Learning

- We can not overwrite the newly computed Q-value with the older value.
- Instead, we use a parameter called learning rate denoted by α , to determine how much information of the previously computed Q-value for the given state-action pair we retain over the newly computed Q-value calculated for the same state-action pair at a later time step.
- The higher the learning rate, the more quickly the agent will adopt the newly computed Q-value.
- Therefore we need to take care of the trade-off between new and old Q-value using the appropriate learning rate.

Q - Learning

$$\text{New } Q(s, a) = Q(s, a) + \alpha [R(s, a) + \gamma \max_{a'} Q'(s', a') - Q(s, a)]$$

- New Q Value for that state and the action
- Learning Rate
- Reward for taking that action at that state
- Current Q Values
- Maximum expected future reward given the new state (s') and all possible actions at that new state.
- Discount Rate

Q – Learning - Example

Q-Table		Actions				
		Action 1	Action 2	...	Action n-1	Action n
States	State 1	0.7334	0.65	...	0.4522	0.6452
	State 2	0.8444	0.4534	...	0.3322	0.7134
	...	...	...	...	...	...
	State n-1	0.7552	0.65	...	0.4522	0.6452
	State n	0.9234	0.4534	...	0.3322	0.7134

Q – Learning Steps

1. **Initialize** all *Q-Values* in the *Q-Table* arbitrary, and the *Q* value of terminal-state to 0:

$$Q(s, a) = n, \forall s \in S, \forall a \in A(s)$$

$$Q(\text{terminal-state}, \cdot) = 0$$

2. **Pick** the action a , from the set of actions defined for that state $A(s)$ defined by the policy π .

3. **Perform** action a

4. **Observe** reward R and the next state s'

5. For all possible actions from the state s' select the one with the **highest** *Q-Value* – a' .

6. **Update** value for the state using the formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma Q(s', a') - Q(s, a)]$$

7. **Repeat** steps 2-5 for each time step until the terminal state is reached

8. **Repeat** steps 2-6 for each episode

Q - Learning

- Q Learning algorithm can be represented as,

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

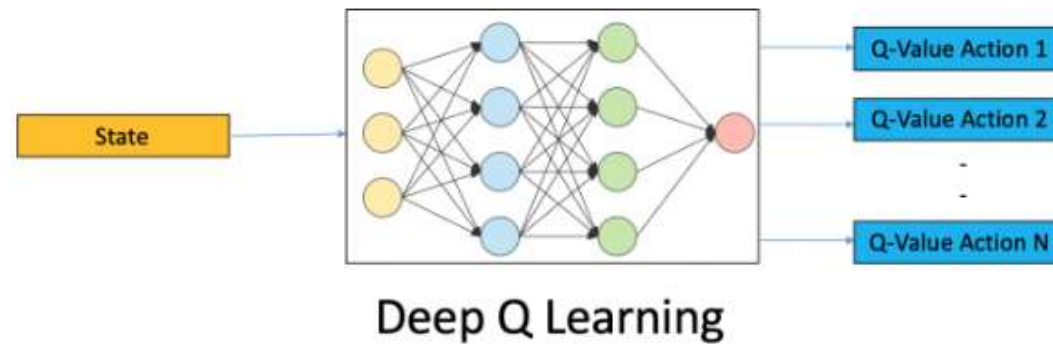
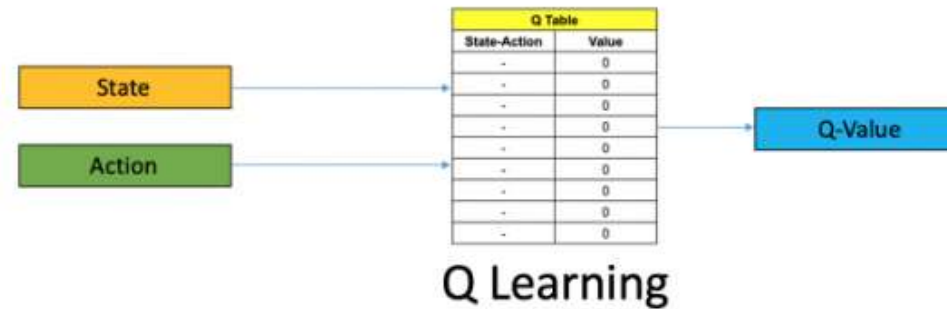
- Where alpha is the learning rate or step size. This simply determines to what extent newly acquired information overrides old information.

Why 'Deep' Q-Learning?

- Q-learning is a simple yet quite powerful algorithm.
- Imagine an environment with 10,000 states and 1,000 actions per state. This would create a table of 10 million cells. Things will quickly get out of control!
- It is pretty clear that we can't infer the Q-value of new states from already explored states. This presents two problems:
 - First, the amount of memory required to save and update that table would increase as the number of states increases
 - Second, the amount of time required to explore each state to create the required Q-table would be unrealistic

Deep Q-Networks

- In deep Q-learning, we use a neural network to approximate the Q-value function. The state is given as the input and the Q-value of all possible actions is generated as the output.



In Deep Q

- All the past experience is stored by the user in memory
- The next action is determined by the maximum output of the Q-network
- The loss function here is mean squared error of the predicted Q-value and the target Q-value – Q^* . This is basically a regression problem.
- However, we do not know the target or actual value here as we are dealing with a reinforcement learning problem.
- Going back to the Q-value update equation derived from the Bellman equation. we have:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[\boxed{R_{t+1} + \gamma \max_a Q(S_{t+1}, a)} - Q(S_t, A_t) \right]$$

- The section in green represents the target.
- We can argue that it is predicting its own value, but since R is the unbiased true reward, the network is going to update its gradient using backpropagation to finally converge.

Pseudocode for Deep Q-Learning

Start with $Q_0(s, a)$ for all s, a .

Get initial state s

For $k = 1, 2, \dots$ till convergence

Sample action a , get next state s'

If s' is terminal:

$$\text{target} = R(s, a, s')$$

Sample new initial state s'

else:

$$\text{target} = R(s, a, s') + \gamma \max_{a'} Q_k(s', a')$$

$$\theta_{k+1} \leftarrow \theta_k - \alpha \nabla_{\theta} \mathbb{E}_{s' \sim P(s'|s,a)} [(Q_{\theta}(s, a) - \text{target}(s'))^2] \big|_{\theta=\theta_k}$$

$$s \leftarrow s'$$

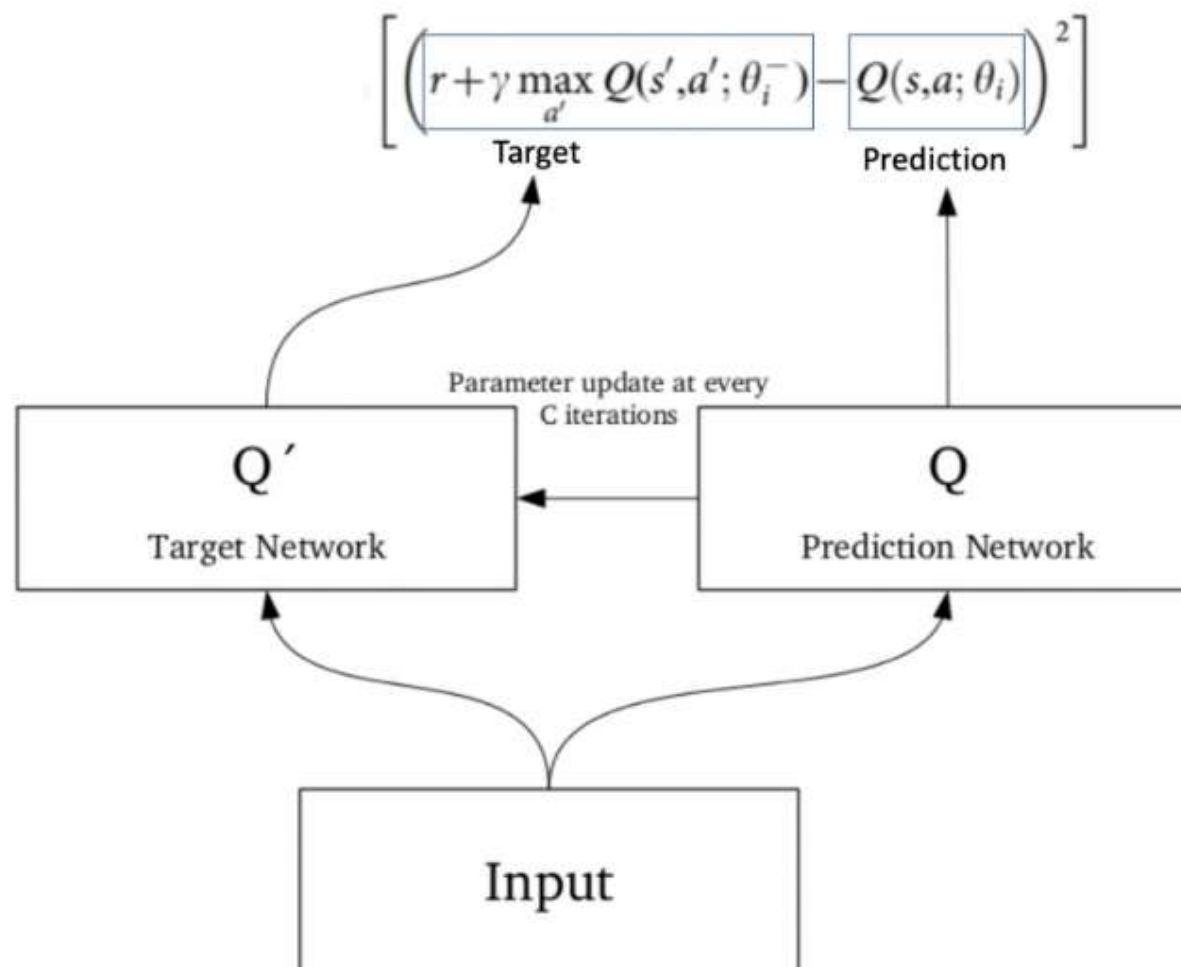
Chasing a nonstationary target!

Updates are correlated within a trajectory!

Target Network

- Since the same network is calculating the predicted value and the target value, there could be a lot of divergence between these two.
- So, instead of using one neural network for learning, we can use two.
- We could use a separate network to estimate the target.
- This target network has the same architecture as the function approximator but with frozen parameters.
- For every C iterations (a hyperparameter), the parameters from the prediction network are copied to the target network.
- This leads to more stable training because it keeps the target function fixed (for a while).

Target Network



Experience Replay

- To perform experience replay, we store the agent's experiences – $et=(st,at,rt,st+1)$
- What does the above statement mean?
- Instead of running Q-learning on state/action pairs as they occur during simulation or the actual experience, the system stores the data discovered for [state, action, reward, next_state] – in a large table.
- Let's understand this using an example.
- Suppose we are trying to build a video game bot where each frame of the game represents a different state. During training, we could sample a random batch of 64 frames from the last 100,000 frames to train our network. This would get us a subset within which the correlation amongst the samples is low and will also provide better sampling efficiency.

Steps of Deep Q-Network

1. Preprocess and feed the game screen (state s) to our DQN, which will return the Q-values of all possible actions in the state
2. Select an action using the epsilon-greedy policy. With the probability epsilon, we select a random action a and with probability $1-\text{epsilon}$, we select an action that has a maximum Q-value, such as $a = \text{argmax}(Q(s,a,w))$
3. Perform this action in a state s and move to a new state s' to receive a reward. This state s' is the preprocessed image of the next game screen. We store this transition in our replay buffer as $\langle s,a,r,s' \rangle$
4. Next, sample some random batches of transitions from the replay buffer and calculate the loss
5. It is known that:

$$\text{Loss} = (r + \gamma \max_{a'} Q(s', a'; \theta') - Q(s, a; \theta))^2$$

which is just the squared difference between target Q and predicted Q

6. Perform gradient descent with respect to our actual network parameters in order to minimize this loss
7. After every C iterations, copy our actual network weights to the target network weights
8. Repeat these steps for M number of episodes